NLP Course
Search documentation Ctrl+K
EN 2,357

0. SETUP
1. TRANSFORMER MODELS
2. USING 🤗 TRANSFORMERS
Introduction
Behind the pipeline
Models
Tokenizers
Handling multiple sequences
Putting it all together
Basic usage completed!
End-of-chapter quiz
3. FINE-TUNING A PRETRAINED MODEL
4. SHARING MODELS AND TOKENIZERS
5. THE 🤗 DATASETS LIBRARY
6. THE 🤗 TOKENIZERS LIBRARY
7. MAIN NLP TASKS
8. HOW TO ASK FOR HELP
9. BUILDING AND SHARING DEMOS NEW
COURSE EVENTS

Join the Hugging Face community
and get access to the augmented documentation experience

Collaborate on models, datasets and Spaces
Faster examples with accelerated inference
Switch between documentation themes

Sign Up to get started

Introduction

Introduction

Ask a question

As you saw in Chapter 1, Transformer models are usually very large. With millions to tens of *billions* of parameters, training and deploying these models is a complicated undertaking. Furthermore, with new models being released on a near-daily basis and each having its own implementation, trying them all out is no easy task.

The 🤗 Transformers library was created to solve this problem. Its goal is to provide a single API through which any Transformer model can be loaded, trained, and saved. The library's main features are:

- **Ease of use**: Downloading, loading, and using a state-of-the-art NLP model for inference can be done in just two lines of code.
- **Flexibility**: At their core, all models are simple PyTorch `nn.Module` or TensorFlow `tf.keras.Model` classes and can be handled like any other models in their respective machine learning (ML) frameworks.
- **Simplicity**: Hardly any abstractions are made across the library. The "All in one file" is a core concept: a model's forward pass is entirely defined in a single file, so that the code itself is understandable and hackable.

This last feature makes 🤗 Transformers quite different from other ML libraries. The models are not built on modules that are shared across files; instead, each model has its own layers. In addition to making the models more approachable and understandable, this allows you to easily experiment on one model without affecting others.

This chapter will begin with an end-to-end example where we use a model and a tokenizer together to replicate the `pipeline()` function introduced in Chapter 1. Next, we'll discuss the model API: we'll dive into the model and configuration classes, and show you how to load a model and how it processes numerical inputs to output predictions.

Then we'll look at the tokenizer API, which is the other main component of the `pipeline()` function. Tokenizers take care of the first and last processing steps, handling the conversion from text to numerical inputs for the neural network, and the conversion back to text when it is needed. Finally, we'll show you how to handle sending multiple sentences through a model in a prepared batch, then wrap it all up with a closer look at the high-level `tokenizer()` function.

> ⚠️ In order to benefit from all features available with the Model Hub and 🤗 Transformers, we recommend creating an account.